

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Технології розподілених систем та паралельних обчислень»

Робота №1

Тема «Основи програмування та засоби синхронізації в багатозадачній
операційній системі»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дьячук Т.С.

МЕТА

Метою роботи є ознайомитися із застосуванням різних методів синхронізації при створенні багатопоточних застосунків.

ЗАВДАННЯ

Завдання на роботу:

1. Відкомпілювати й запустити запропонований приклад простий консольної багатопоточної комп'ютерної гри «Війна програмних потоків».
2. Розібрати текст програми ThreadWar.cpp. Знайти й підкреслити в тексті програми функції WinAPI для роботи з об'єктами синхронізації. Пояснити їх застосування.

ВИКОНАННЯ

Текст програми ThreadWar.cpp

```
// Проста комп'ютерна гра Thread War
// Використовуйте клавіші "уліво" і "вправо", щоб переміщати пушку
// клавіша "пробіл" робить постріл,
// Якщо 30 ворогів підуть із екрана не знищеними, ви програли
// Очки даються за кожного вбитого супротивника

#include <windows.h>
#include <process.h>
#include <stdlib.h>
#include <time.h>
#include <stdio.h>

// Об'єкти синхронізації
HANDLE screenlock;    // зміною екрана займається тільки один потік
HANDLE bulletsem;     // можна вистрілити тільки три рази підряд
HANDLE startevt;      // гра починається з натисканням клавіші "уліво"
або "вправо"
HANDLE conin, conout; // дескриптори консолі
HANDLE mainthread;    // основний потік main
CRITICAL_SECTION gameover;
```

```

CONSOLE_SCREEN_BUFFER_INFO info; // інформація про консоль
// кількість влучень і промахів
long hit = 0;
long miss = 0;
long delayfactor = 7; // фактор затримки для ворогів

// Створення випадкового числа від n0 до n1
int random(int n0, int n1)
{
    if (n0 == 0 && n1 == 1)
        return rand() % 2; // спеціальний випадок
    return rand() % (n1 - n0) + n0;
}

// Очищення екрана консолі
void cls()
{
    COORD org = {0, 0};
    DWORD res;
    FillConsoleOutputCharacter(conout, ' ', info.dwSize.X *
info.dwSize.Y, org, &res);
}

// вивести на екран символ в позицію x і y
void writeat(int x, int y, char c)
{
    // Блокувати вивід на екран за допомогою м'ютекса
    WaitForSingleObject(screenlock, INFINITE);
    COORD pos = {x, y};
    DWORD res;
    WriteConsoleOutputCharacter(conout, &c, 1, pos, &res);
    ReleaseMutex(screenlock);
}

// Одержати натискання на клавішу (лічильник повторень в ct)
int getakey(int &ct)
{
    INPUT_RECORD input;
    DWORD res;
    while (1)
    {
        ReadConsoleInput(conin, &input, 1, &res);

        // ігнорувати інші події
        if (input.EventType != KEY_EVENT)
            continue;

        // ігнорувати події відпускання клавіш
        // нас цікавлять тільки натискання
        if (!input.Event.KeyEvent.bKeyDown)
            continue;
        ct = input.Event.KeyEvent.wRepeatCount;
        return input.Event.KeyEvent.wVirtualKeyCode;
    }
}

// Обробка комбінацій ^C, ^Break, і т.і.
BOOL WINAPI ctrl(DWORD type)

```

```

{
    exit(0);
    return TRUE;
    // не досяжна ділянка коду
}

// Визначити символ в заданій позиції екрана
int getat(int x, int y)
{
    char c;
    DWORD res;
    COORD org = {x, y};

    // Блокувати доступ до консолі доти, поки процедура не буде
    виконана
    WaitForSingleObject(screenlock, INFINITE);
    ReadConsoleOutputCharacter(conout, &c, 1, org, &res);
    ReleaseMutex(screenlock); // unlock
    return c;
}

// Відобразити очки в заголовку вікна й перевірити умову завершення гри
void score(void)
{
    char s[128];
    sprintf(s, "Thread War! Hit: %d Miss : %d", hit, miss);
    SetConsoleTitle(s);
    if (miss >= 30)
    {
        EnterCriticalSection(&gameover);
        SuspendThread(mainthread); // призупинити головний потік
        MessageBox(NULL, "Game Over!", "Thread War", MB_OK |
MB_SETFOREGROUND);
        exit(0); // не виходить із критичної секції
    }
    if ((hit + miss) % 20 == 0)
        InterlockedDecrement(&delayfactor); // повинен бути ilock
}
char badchar[] = "-\\|/";
// це потік супротивника
void badguy(void *_y)
{
    int y = (int)_y; // випадкова координата y
    int dir;
    int x;
    // непарні y з'являються ліворуч, парні y з'являються праворуч
    x = y % 2 ? 0 : info.dwSize.X;
    // установити напрямок залежно від початкової позиції
    dir = x ? -1 : 1;
    // поки супротивник перебуває в межах екрана
    while ((dir == 1 && x != info.dwSize.X) || (dir == -1 && x != 0))
    {
        int dly;
        BOOL hitme = FALSE;
        // перевірка на влучення (куля?)
        if (getat(x, y) == '*')
            hitme = TRUE;
    }
}

```

```

        // вивід символу на екран
        writeat(x, y, badchar[x % 4]);
        // ще одна перевірка на влучення
        if (getat(x, y) == '*')
            hitme = TRUE;

        // перевірка на влучення через невеликі
        // проміжки часу
        if (delayfactor < 3)
            dly = 3;
        else
            dly = delayfactor + 3;
        for (int i = 0; i < dly; i++)
        {
            Sleep(40);
            if (getat(x, y) == '*')
            {
                hitme = TRUE;
                break;
            }
        }
        writeat(x, y, ' ');
        // ще одна перевірка на влучення
        if (getat(x, y) == '*')
            hitme = TRUE;
        if (hitme)
        {
            // у супротивника влучили!
            MessageBeep(-1);
            InterlockedIncrement(&hit);
            score();
            _endthread();
        }
        x += dir;
    }
    // супротивник утік!
    InterlockedIncrement(&miss);
    score();
}

// цей потік займається створенням потоків супротивників
void badguys(void *)
{
    // чекаємо сигналу до початку гри протягом 15 секунд
    WaitForSingleObject(startevt, 15000);
    // створюємо випадкового ворога
    // кожні 5 секунд з'являється шанс створити
    // супротивника з координатами від 1 до 10
    while (1)
    {
        if (random(0, 100) < (hit + miss) / 25 + 20)
        {
            // згодом імовірність збільшується
            _beginthread(badguy, 0, (void *) (random(1, 10)));
            Sleep(1000); // щосекунди
        }
    }
}

```

```

// Це потік кулі
// кожна куля - це окремий потік
void bullet(void *_xy_)
{
    COORD xy = *(COORD *)_xy_;
    if (getat(xy.X, xy.Y) == '*')
        return; // тут уже є куля
    // треба почекати
    // перевірити семафор
    // якщо семафор дорівнює 0, пострілу не відбувається
    if (WaitForSingleObject(bulletsem, 0) == WAIT_TIMEOUT)
        return;
    while (-ixy.Y)
    {
        writeat(xy.X, xy.Y, '*'); // відобразити кулю
        Sleep(100);
        writeat(xy.X, xy.Y, ' '); // стерти кулю
    }
    // постріл зроблений - додати 1 до семафора
    ReleaseSemaphore(bulletsem, 1, NULL);
}

// Основна програма
void main()
{
    HANDLE me;
    // Налаштування глобальних змінних
    conin = GetStdHandle(STD_INPUT_HANDLE);
    conout = GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleCtrlHandler(ctrl, TRUE);
    SetConsoleMode(conin, ENABLE_WINDOW_INPUT);
    me = GetCurrentThread(); // не є реальним дескриптором

    // змінити псевдодескриптор на реальний дескриптор поточного потоку
    DuplicateHandle(GetCurrentProcess(), me, GetCurrentProcess(),
&mainthread, 0, FALSE, DUPLICATE_SAME_ACCESS);
    startevt = CreateEvent(NULL, TRUE, FALSE, NULL);
    screenlock = CreateMutex(NULL, FALSE, NULL);
    InitializeCriticalSection(&gameover);
    bulletsem = CreateSemaphore(NULL, 3, 3, NULL);
    GetConsoleScreenBufferInfo(conout, &info);

    // Ініціалізувати відображення інформації про очки
    score();
    // Налаштувати генератор псевдовипадкових чисел
    srand((unsigned)time(NULL));
    cls(); // насправді не потрібно
    // установка початкової позиції пушки
    int y = info.dwSize.Y - 1;
    int x = info.dwSize.X / 2;
    // запустити потік badguys; нічого не робити доти,
    // поки не відбудеться подія або минуть 15 секунд
    _beginthread(badguys, 0, NULL);
    // основний цикл гри
    while (1)
    {
        int c, ct;
        writeat(x, y, '|'); // намалювати пушку
    }
}

```

```

c = getakey(ct);    // одержати символ
switch (c)
{
case VK_SPACE: // вогонь!
{
    static COORD xy;
    xy.X = x;
    xy.Y = y;

    _beginthread(bullet, 0, (void *)&xy);
    Sleep(100); // дати кулі час полетіти на деяку відстань
    break;
}
case VK_LEFT:      // команда "уліво!"
    SetEvent(startevt); // потік badguys працює
    writeat(x, y, ' '); // зтерти з екрана пушку
    while (ct--)      // переміститися
        if (x)
            x--;
    break;
case VK_RIGHT: // команда "вправо!"; логіка та ж
    SetEvent(startevt);
    writeat(x, y, ' ');
    while (ct - i)
        if (x != info.dwSize.X - 1)
            x++;
    break;
}
}
}

```

Схема взаємодії потоків

Схему взаємодії потоків наведено на рисунку нижче:

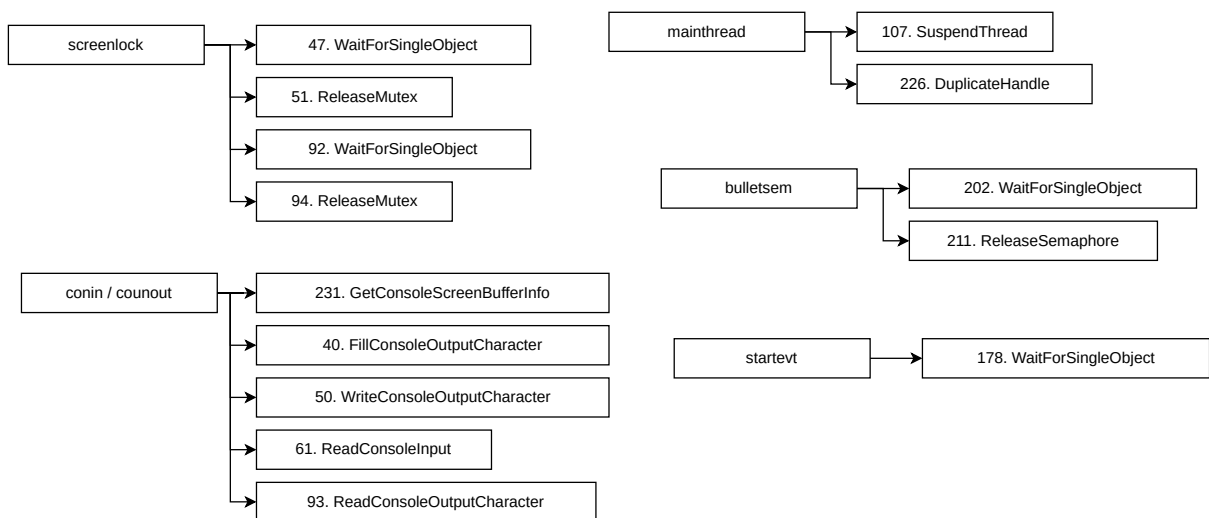


Рисунок 1.1 — Схема взаємодії потоків

Опис функцій Windows API, що зустрічаються в програмі

Описи функцій, знайдених у програмі, наведено нижче на таблиці:

Таблиця 1.1 — Функції Windows API, знайдені у програмі

Рядок	Функція	Призначення	Параметри	Значення, що повертаються
47, 92	WaitForSingleObject	Очікує поки вказаний об'єкт перейде в сигнальний стан або мине інтервал очікування.	Screenlock, INFINITE	При успішному виконанні функції повернуте значення вказує на подію, що стала причиною повернення результату. Може приймати такі значення: WAIT_ABANDONED, WAIT_OBJECT_0, WAIT_TIMEOUT, WAIT_FAILED
178			Startevt, 15000	
202			Bulletsem, 0	
51, 94	ReleaseMutex	Звільняє право власності на вказаний об'єкт mutex.	Screenlock	При успішному виконанні повертає ненульове значення. При неуспішному виконанні повертає нульове значення
106	EnterCriticalSection	Чекає на власника вказаного об'єкта критичної секції. Функція повертається, коли потоку, що викликає, було надано право власності.	&gameover	Не повертає значення, викликає EXCEPTION_POSSIBLE_DEADLOCK
107	SuspendThread	Призупиняє вказаний потік	Mainthread	При успішному виконанні функція повертає значення попереднього лічильника зупиненого потоку. Інакше — (DWORD) - 1
187	_beginthread	Створює потік	Badguy, 0, random(1,10)	При успішному виконанні повертає дескриптор новоствореного потоку.
243			Badguy, 0	
258			Bullet, 0, &xy	
211	ReleaseSemaphore	Збільшує кількість вказаного семафорного об'єкта на задану величину	Bulletsem, 1	При успішному виконанні повертає нуль.
206, 208	Writeat	Записує задану кількість байтів,	xy.X, xy.Y	Повертає одне з наступних значень:

Рядок	Функція	Призначення	Параметри	Значення, що повертаються
		починаючи з заданого зсуву.		- S_OK; - E_FAIL; - E_PENDING; - STG_E_ACCESSDENIED; - STG_E_WRITEFAULT; - STG_E_MEDIUMFULL.
223	GetCurrentThread	Отримує псевдо-дескриптор для потоку, що викликає.	-	Не повертає значення.
226	DuplicateHandle	Дублює дескриптор об'єкта	GetCurrentProcess, GetCurrentProcess, mainthread, 0	При успішному виконанні повертає нуль.
227	CreateEvent	Створює або відкриває іменованний або безіменний об'єкт події.	True, False	При успішному виконанні повертає дескриптор об'єкта події.
228	CreateMutex	Створює або відкриває іменованний або безіменний mutex-об'єкт.	False	При успішному виконанні повертає дескриптор створеного mutex-об'єкта.
229	InitializeCriticalSection	Ініціалізує об'єкт критичного розділу.	&gameover	Не повертає значення.
230	CreateSemaphore	Створює або відкриває іменованний або безіменний семафорний об'єкт.	3, 3	При успішному виконанні повертає дескриптор об'єкту семафору.

ПИТАННЯ

Поняття процесу і потоку. Стани процесу.

Процес — екземпляр програми, що виконується.

Потік — послідовність інструкцій, яка виконується всередині процесу.

Стани процесу наведено на таблиці нижче:

Таблиця 2.1 — Стани процесу

Назва	Опис
Створений (<i>New / Created</i>)	Процес створено системою.
Готовий (<i>Ready</i>)	Процес завантажено у пам'ять.
Виконуваний (<i>Running</i>)	Інструкції процесу активно виконуються на процесорі.
Заблокований (<i>Blocked / Waiting</i>)	Процес не може бути виконаний, бо чекає на подію (наприклад, завершення введення / виведення, звільнення ресурсу)
Завершений (<i>Terminated / Exit</i>)	Процес припинив роботу та звільнив ресурси

Багатозадачність. Багатопроцесорність.

Багатозадачність — здатність операційної системи одночасно обробляти кілька процесів.

Багатопроцесорність — використання двох чи більше ядер процесора в одній системі для одночасного виконання різних частин програми (або декількох програм).

Моделі взаємодії процесів. Конкуренція.

Моделі взаємодії процесів — механізм, який дозволяє процесам обмінюватися даними та синхронізувати свої дії.

Конкуренція — здатність системи обробляти кілька завдань одночасно.